Assignment 3

Course	Programming Fundamentals
Instructor	Ms. Hina Iqbal
Section	BCS-1E, BCS-1F

Important Guidelines:

1. If you use **AI tools** (ChatGPT, Copilot, etc.) or **copy code from other students/online sources**, your assignment will be given **zero marks** without exception.

   Prophet Muhammad (SAW) said: "Truthfulness leads to righteousness, and righteousness leads to Paradise" - *Sahih Al Bukhari 6094*

2. Submit your assignment **on time**. Late submissions will incur a 25%-mark deduction if submitted within 6 hours, a 50%-mark deduction if submitted within 24 hours, and submissions beyond 24 hours will result in a score of zero.
3. No marks will be awarded to individuals who do not submit the file on Google Classroom (GCR).

4. Only submit your cpp files on google classroom. **Do not zip your files.** Each cpp file should be renamed according to the format: **l25xxxx_q1.cpp. Failure to follow this format will result in a score of zero.**
5. In case of confusion please feel free to contact on email ***l227910@lhr.nu.edu.pk*** **or** ***hina.iqbal@nu.edu.pk***

Questions

Q1. Scientific Number Transformer

Write an overloaded function named `transform()` with the following behaviors:

1. `double transform(double x)`
   - Returns the value of: **$3x^2 - 5x + 12$**
2. `double transform(double base, int power)`
   - Returns **base^power**
3. `double transform(double values[], int size)`
   - Returns the **average** of all numbers in the array.

Input Example

```
Input:
x = 3
base = 2, power = 5
N = 4
Array: 4 8 12 16
```

Output Example

```
Expression result: 24
Power result: 32
Average: 10
```

Edge Cases to Handle

- Negative powers
- Empty array (size 0 → print error)

Q2. Modular Student Report Generator

A university stores student marks in an array of size **N (1–100)**.

Write a program using **functions only** that:

1. Inputs marks of students.
2. Calculates:
    - Highest
    - Lowest
    - Average
    - Standard deviation
    - Number of students above average
3. Displays a report in a formatted table.

Input Example

```
5
60 70 80 90 100
```

Output Example

```
Highest: 100
Lowest: 60
Average: 80
Std Dev: 14.14
Above Average: 2
```

Q3. Delivery Route Analyzer

A logistics company stores daily distances travelled by a delivery truck in a 1D array.

Requirements

- Input: N distances (1 ≤ N ≤ 200)
- Using **functions only**, compute:
    - Longest increasing distance streak
    - Longest decreasing distance streak
    - Total fuel cost *(fuel cost per km = Rs. 270)*
    - Number of days where distance exceeded **average by at least 20%**

Input Example

```
8
20 25 30 28 26 27 29 40
```

Output Example

```
Longest Increasing Streak: 3 days
Longest Decreasing Streak: 2 days
Fuel Cost: Rs. 54840
```

```
Above Avg +20% Days: 1
```

Q4. Secure ID Validator

You must validate employee IDs of the format:
ABC-1234-X,
where:

- First 3 chars → uppercase alphabets
- Hyphen
- 4 digits
- Hyphen
- Last char → uppercase alphabet

Write a C++ program **using only character arrays and manual checks** (no `string` functions like `isdigit`).

Input Example

```
Enter ID: HRM-4921-K
```

Output Example

```
Valid ID
```

Invalid Example

```
Invalid ID: Last character must be A-Z
```

Edge Cases

- Wrong length
- Missing hyphens
- Lowercase letters
- Non-numeric characters

Q5. Set Operations using Arrays – Union, Intersection, Difference

Given two **unsorted** integer arrays:

- Array A of size N
- Array B of size M

Write functions to compute:

1. **Union** (no duplicates)
2. **Intersection**
3. **A − B Difference**

Arrays may contain duplicates; results must not.

Input Example

```
A = {4, 4, 7, 9}
B = {7, 1, 4}
```

Output Example

```
Union: 4 7 9 1
Intersection: 4 7
```

```
Difference (A-B): 9
```

Constraints

- Implement your own duplicate-removal logic
- No built-in STL

Q6. Student Quiz Matrix Processor

A class records quiz marks in a **2D array**, where:

- Rows = students
- Columns = quizzes

Write a program that uses **functions** to:

1. Input matrix
2. Compute row-wise (student-wise):
    - Min
    - Max
    - Average
3. Compute column-wise (quiz-wise):
    - Highest
    - Lowest
4. Identify the student with the **highest total score**

Input Example

```
3 3
10 8 9
7 9 6
9 9 9
```

Output Example

```
Student 1 -> Min: 8 Max: 10 Avg: 9
Student 2 -> Min: 6 Max: 9 Avg: 7.33
Student 3 -> Min: 9 Max: 9 Avg: 9

Quiz 1 -> Min: 7 Max: 10
Quiz 2 -> Min: 8 Max: 9
Quiz 3 -> Min: 6 Max: 9

Top Scorer: Student 3
```

Edge Case

- Marks outside 0–10 → invalid

Q7. Matrix Operations – Addition, Multiplication, Transpose

Using **2D arrays** and **separate functions**, implement:

- Matrix addition
- Matrix multiplication
- Transpose of first matrix

Matrix size: up to **5×5**

Input Example

```
Matrix A 2x2:
1 2
3 4

Matrix B 2x2:
5 6
7 8
```

Output Example

```
Addition:
6 8
10 12

Multiplication:
19 22
43 50

Transpose of A:
1 3
2 4
```

Edge Cases

- Non-conformable matrices for multiplication
- Invalid matrix sizes

Q8. Rotation & Shifting – Warehouse Storage Simulator

A warehouse stores items on a **1D shelf array** of size N.

Required Operations (each must be a separate function):

- Right rotation by k positions
- Left rotation by k positions
- Remove an item from a given index (shift left)
- Insert an item at a given index (shift right)

Input Example

```
N = 6
Shelf: 3 9 1 7 4 8
Rotate Right by 2
Insert 5 at index 3
Remove item at index 0
```

Output Example

```
After Right Rotation: 4 8 3 9 1 7
After Insert: 4 8 3 5 9 1 7
After Removal: 8 3 5 9 1 7
```

Edge Cases

- Insert or remove at invalid index

Q9. Real-World Scheduling – Classroom Timetable Validator

A university assigns **classroom schedules** represented by a 2D array:

- Rows → time slots
- Columns → rooms

- Each cell stores a **course code** or `"--"` for empty.

Write a program to validate:

1. No consecutive classes of the same course in the same room (column conflict)
2. No course appears **twice in the same time slot** across rooms
3. Count:
    - Total empty slots
    - Most frequently scheduled course

Input Example

```
3 4
CS101  MTH102 --     CS101
--     --     ENG201 MTH102
MGT200 MTH102 --     --
```

Output Example

```
Conflict: CS101 appears twice in Slot 1
Total Empty Slots: 5
Most Frequent Course: MTH102
```

Q10. Longest Balanced Submatrix

A **balanced submatrix** is defined as a contiguous block of the 2D array where:

- Number of even elements = number of odd elements
- Submatrix must be at least **2×2**

Write a program that:

1. Inputs an R×C integer matrix
2. Determines the **largest balanced submatrix**
3. Outputs only the **area** (rows × columns)

Example Input

```
4 4
2 3 4 1
6 5 8 10
3 7 2 4
9 1 6 2
```

Example Output

```
Largest Balanced Submatrix Area: 6
```

Edge Cases

- No balanced submatrix → output 0
- Matrix with all even/all odd → 0
- Multiple submatrices → return largest-area only

GOOD LUCK